

Project Summary: AI-Powered RFQ Data Extraction System

1. Overview This project implements an end-to-end solution for extracting structured data from vendor RFQ (Request for Quotation) documents (.doc → PDF), enriching it with AI-driven parsing, and storing in MongoDB. The system comprises frontend, backend, AI model service, and database—all hosted on a Contabo VPS.

2. Tech Stack

- **Frontend:** React (or Next.js) for user interface and PDF upload
- **Backend:** Node.js with Express framework
- **AI/LLM Engine:** Ollama (running local LLaMA3 or Mistral model)
- **PDF Processing:** `pdf-parse` or `pdf2json` for text & layout extraction
- **Database:** MongoDB (Mongoose ODM)
- **Hosting:** Contabo VPS (8–16 GB RAM, Ubuntu 22.04)
- **File Storage:** Local `uploads/` directory or GridFS for drawings

3. Hosting & Deployment on Contabo VPS

1. **Server Setup**
2. Ubuntu 22.04
3. Node.js 18.x, Git, tmux, curl
4. **Ollama Installation**

```
curl -fsSL https://ollama.com/install.sh | sh
ollama pull llama3
tmux
ollama run llama3
```

5. **Backend Deployment**
6. Clone repo, `npm install express multer pdf-parse axios`
7. `node server.js` or `pm2 / docker-compose`
8. **MongoDB**
9. `apt install mongodb` or Docker `mongo` container
10. **Reverse Proxy (optional)**
11. NGINX to route `/api` → `localhost:3000`, serve static frontend

4. Key Features

- **PDF Upload & Parsing:** Users upload PDFs; backend extracts raw text.
- **AI-Powered Field Extraction:** Local Ollama processes prompt to extract fields:
 - `item_code`, `item_name`, `drawing_number`, `quantity`, `unit`.
- **Detailed Schemas:** Each RFQ contains multiple items with:
 - **Basic Details:** serial, itemCode, name, qty, uom
 - **Technical Details:** MOC, hardness, surfaceFinish, heatTreatment
 - **Commercial Details:**
 - **Raw Materials:** name, rate, weight, totalCost
 - **Labour Operations:** operationName, cost
 - marginPercent, additionalCharges, costPrice, sellPrice, taxPercent, freightCharge

- **Storage & Queries:** MongoDB schema optimized for single-document reads and indexing on `items.itemCode`.

5. Data Models & Schemas (MongoDB / Mongoose)

```
// Technical specs
const TechnicalSchema = new Schema({ MOC: String, hardness: String,
surfaceFinish: String, heatTreatment: String }, { _id: false });

// Labour and material cost breakdown
const LabourOperationSchema = new Schema({ operationName: String, cost:
Number }, { _id: false });
const RawMaterialSchema = new Schema({ name: String, rate: Number, weight:
Number, totalCost: Number }, { _id: false });

// Commercial details
const CommercialSchema = new Schema({ rawMaterials: [RawMaterialSchema],
labourOperations: [LabourOperationSchema], marginPercent: Number,
additionalCharges: Number, costPrice: Number, sellPrice: Number, taxPercent:
Number, freightCharge: Number }, { _id: false });

// Line item
const ItemSchema = new Schema({ serial: Number, itemCode: String, name:
String, qty: Number, uom: String, drawingUrl: String, technical:
TechnicalSchema, commercial: CommercialSchema }, { _id: false });

// RFQ document
const RFQSchema = new Schema({ title: String, owner: String, description:
String, createdAt: Date, dueDate: Date, status: { enum:
["open", "closed", "awarded"] }, items: [ItemSchema] });
RFQSchema.index({ "items.itemCode": 1 });
```

6. Libraries & Tools

- `express`, `multer` (file upload)
- `pdf-parse` / `pdf2json` (PDF → text/layout)
- `axios` (HTTP to Ollama API)
- `mongoose` (MongoDB ODM)
- `ollama` (local AI inference)
- `tmux` or `pm2` (process management)
- `nginx` + `certbot` (optional reverse proxy & SSL)

7. Next Steps & Extras

- Add file-upload UI in React/Next.js
- Automate Docker-compose for all services
- Set up NGINX reverse proxy with Let's Encrypt SSL
- Implement role-based auth for users (admin, vendor)
- Extend AI prompts for more fields or scanned PDFs (Tesseract OCR)

End of Project Summary.